Escola de Artes, Ciências e Humanidades — Universidade de São Paulo

RELATÓRIO DE ESTUDO

Estudo da Família de Algoritmos Ascon

Área: Sistemas de Informação / Segurança da Informação
Estudante: Eduardo Diniz da Silva
Orientador: Prof. Fábio Nakano

1 Resumo

Ascon é uma família de algoritmos de criptografia leve, desenvolvida por Dobraunig, Eichlseder, Mendel e Schläffer, com o objetivo de oferecer eficiência e facilidade de implementação. A família Ascon foi selecionada pelo NIST (*National Institute of Standards and Technology*) como padrão para criptografia autenticada leve e hashing, após uma competição de criptografia leve realizada em 2023.

A família Ascon caracteriza-se pela leveza e pelo uso de primitivas baseadas em permutações para prover segurança, robustez, eficiência e flexibilidade. Dessa forma, mostra-se adequada para ambientes com recursos limitados, como dispositivos IoT (*Internet of Things*).

2 Contexto e Conceitos Iniciais

Chave Secreta: Podemos compreender o conceito de uma chave secreta K por meio do Princípio de Kerckhoffs. Esse princípio estabelece que um sistema criptográfico pode ser conhecido por todos e, ainda assim, permanecer seguro, desde que utilize uma chave desconhecida. [1]

Desse modo, a segurança de um sistema deve depender apenas do sigilo da chave, e não do sigilo do algoritmo ou de seu projeto. Assim, podemos definir uma chave secreta, segundo o NIST, como um parâmetro utilizado em conjunto com um algoritmo de criptografia para validar, computar ou verificar uma transformação criptográfica. [2]

Em alguns sistemas criptográficos, esse conceito se expande e pode envolver um par de chaves matematicamente complementares: a chave pública e a chave privada. Bruce Schneier explica esse par, em seu livro *Applied Cryptography*, por meio de uma analogia com uma caixa de correio:

> "Colocar uma correspondência na caixa de correio é análogo a criptografar com a chave pública; qualquer um pode fazer isso. Basta abrir a fresta e soltá-la lá dentro. Retirar a correspondência de uma caixa de correio é análogo a descriptografar com a chave privada. Geralmente é difícil; você precisa de maçaricos." [4]

A ideia é que, em um criptossistema de chave pública, se uma mensagem for criptografada com a chave pública de alguém, apenas a chave privada correspondente dessa pessoa poderá descriptografá-la.

3 Estrutura Interna

Todos os membros da família Ascon utilizam a mesma permutação leve. Essa permutação segue uma estrutura de esponja conhecida como *sponge construction*. A estrutura de esponja ilustra a ideia de "absorver" dados, processá-los internamente e, posteriormente, "espremê-los" para produzir o resultado.

Para isso, a estrutura divide-se em três fases principais: inicialização, processamento de dados e finalização.

Na inicialização, o algoritmo define seu estado interno de 320 bits com as configurações do sistema. No processamento, os dados são "absorvidos" em cinco fatias de tamanho fixo da taxa do algoritmo, e o algoritmo executa operações de mistura após cada bloco, a fim de combinar o estado atual com o anterior.

Após as rodadas de mistura, na finalização, o sistema executa um último embaralhamento e "espreme" o estado interno para produzir o resultado final em um tamanho padronizado. [5]

4 Permutação

A permutação utilizada pelos membros da família Ascon é referenciada por Ascon-$p[rnd]$, em que p é a função de permutação e rnd é o número de rodadas de permutação.

Ela segue a arquitetura de Rede de Substituição-Permutação (SPN) e divide-se em três etapas:

$$p = p_L \circ p_S \circ p_C \tag{1}$$

Antes do início das três etapas, ocorre a definição do estado interno.

Estado Interno: A permutação processa um estado total S de 320 bits. Esse estado é dividido em cinco palavras de 64 bits, denotadas por S_i:

$$S = S_0 \parallel S_1 \parallel S_2 \parallel S_3 \parallel S_4 \tag{2}$$

Cada palavra atua como um número inteiro sem sinal de 64 bits, posicionado com o bit menos significativo à direita, ou seja, organizado do maior para o menor valor da esquerda para a direita.

Após a definição, ocorrem as três etapas da permutação.

Camada de Adição de Constante (p_C)

A constante de rodada c_i da permutação Ascon-$p[rnd]$, sendo $rnd \leq 16$ e $0 \leq i \leq rnd - 1$, é definida como:

$$c_i = const_{16-rnd+i} \tag{3}$$

em que $const_i$ segue a seguinte tabela:

Tabela 1: Constantes $const_i$ utilizadas nas rodadas das permutações Ascon

i	$const_i$	i	$const_i$
0	0x000000000000003c	8	0x00000000000000b4
1	0x000000000000002d	9	0x00000000000000a5
2	0x000000000000001e	10	0x0000000000000096
3	0x000000000000000f	11	0x0000000000000087
4	0x00000000000000f0	12	0x0000000000000078
5	0x00000000000000e1	13	0x0000000000000069
6	0x00000000000000d2	14	0x000000000000005a
7	0x00000000000000c3	15	0x000000000000004b

A Camada de Adição de Constante p_C adiciona uma constante de rodada c_i à palavra S_2 por meio da operação XOR na rodada i, para $i \geq 0$. Como os 56 primeiros bits da constante são zeros, na prática, essa adição afeta apenas os 8 bits menos significativos de S_2.

Camada de Substituição (p_S)

A Camada de Substituição p_S atualiza o estado S com 64 aplicações paralelas da caixa de substituição de 5 bits, denominada SBox:

$$(s_{(0,j)}, s_{(1,j)}, \ldots, s_{(4,j)}) = \text{SBOX}(s_{(0,j)}, s_{(1,j)}, \ldots, s_{(4,j)}) \tag{4}$$

para todo $0 \leq j < 64$.

A SBox é calculada da seguinte maneira:

$$(y_0, \ldots, y_4) = \text{SBOX}(x_0, \ldots, x_4) \tag{5}$$

em que:

$$y_0 = x_4x_1 \oplus x_3 \oplus x_2x_1 \oplus x_2 \oplus x_1x_0 \oplus x_1 \oplus x_0 \tag{6}$$
$$y_1 = x_4 \oplus x_3x_2 \oplus x_3x_1 \oplus x_3 \oplus x_2x_1 \oplus x_2 \oplus x_1 \oplus x_0 \tag{7}$$
$$y_2 = x_4x_3 \oplus x_4 \oplus x_2 \oplus x_1 \oplus 1 \tag{8}$$
$$y_3 = x_4x_0 \oplus x_4 \oplus x_3x_0 \oplus x_3 \oplus x_2 \oplus x_1 \oplus x_0 \tag{9}$$
$$y_4 = x_4x_1 \oplus x_4 \oplus x_3 \oplus x_1x_0 \oplus x_1 \tag{10}$$

Ou seja, a SBox é uma função que recebe cinco bits independentes de entrada $(x_0, \ldots, x_4)$, sendo cada bit j de cada uma das cinco palavras, e devolve cinco bits transformados $(y_0, \ldots, y_4)$ por meio de operações do tipo:

- **AND:** representada pela multiplicação de variáveis. Exemplo: x_4x_1.
- **XOR:** representada pelo operador bit a bit ou exclusivo $\oplus$. Exemplo: $x_4 \oplus x_2$.
- **NOT:** representada pela operação XOR com 1. Exemplo: $x_1 \oplus 1$.

Além disso, como a entrada possui apenas cinco bits, existem apenas $2^5 = 32$ resultados possíveis. Desse modo, pode-se utilizar uma tabela de consulta:

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
SBox(x)	4	b	1f	14	1a	15	9	2	1b	5	8	12	1d	3	6	1c

x	10	11	12	13	14	15	16	17	18	19	1a	1b	1c	1d	1e	1f
SBox(x)	1e	13	7	e	0	d	11	18	10	c	1	19	16	a	f	17

Tabela 2: Tabela de consulta da SBox do Ascon.

Note que as entradas de cinco bits são representadas em hexadecimal. Por exemplo, $x = 1$ corresponde a $(0, 0, 0, 0, 1)$.

Essa lógica matemática pode ser facilmente aplicada em um circuito. O documento de especificação do Ascon detalha como as portas lógicas (XOR, AND e NOT) são fisicamente encadeadas para isso. Essa facilidade torna o algoritmo eficiente para execução em microchips com baixo consumo de energia.

Camada de Difusão Linear (p_L)

A Camada de Difusão Linear p_L provê difusão entre cada uma das palavras de 64 bits S_i de maneira horizontal. Diferentemente da Camada de Substituição, que atua verticalmente, a Camada de Difusão embaralha cada palavra S_i de forma independente. Assim, o que acontece em S_0 não interfere em S_1.

Essa camada aplica as funções lineares Σ_i às respectivas palavras, de modo que $S_i \leftarrow \Sigma_i(S_i)$ para $0 \leq i \leq 4$, em que cada Σ_i é definida como:

$$\Sigma_0(S_0) = S_0 \oplus (S_0 \ggg 19) \oplus (S_0 \ggg 28) \tag{11}$$
$$\Sigma_1(S_1) = S_1 \oplus (S_1 \ggg 61) \oplus (S_1 \ggg 39) \tag{12}$$
$$\Sigma_2(S_2) = S_2 \oplus (S_2 \ggg 1) \oplus (S_2 \ggg 6) \tag{13}$$
$$\Sigma_3(S_3) = S_3 \oplus (S_3 \ggg 10) \oplus (S_3 \ggg 17) \tag{14}$$
$$\Sigma_4(S_4) = S_4 \oplus (S_4 \ggg 7) \oplus (S_4 \ggg 41) \tag{15}$$

Note que o operador $\ggg$ indica rotação à direita, ou seja, desloca os 64 bits da palavra para a direita. Os bits que "caem" da extremidade direita dão a volta e reaparecem na extremidade esquerda.

Desse modo, o algoritmo combina a palavra original com duas cópias dela mesma que foram rotacionadas.

5 Principais Algoritmos

Nesta seção, serão apresentados dois algoritmos da família Ascon e suas respectivas especificações: Ascon-AEAD128 e Ascon-Hash256.

5.1 Ascon-AEAD128

O Ascon-AEAD128 é o principal algoritmo e a recomendação padrão da família para Criptografia com Dados Associados (AEAD). Ele consiste em um algoritmo de criptografia, `Ascon-AEAD128.enc`, e um algoritmo de descriptografia, `Ascon-AEAD128.dec`.

O `Ascon-AEAD128.enc` utiliza uma chave secreta K de 128 bits, um nonce N de 128 bits, um dado associado A de tamanho variável e um texto plano P de tamanho variável como entradas. Como saída, retorna um texto cifrado C, com o mesmo tamanho de P, e uma tag de autenticação T de 128 bits.

$$\text{Ascon-AEAD128.enc}(K, N, A, P) = (C, T) \tag{16}$$

O `Ascon-AEAD128.dec` utiliza uma chave K, um nonce N, um dado associado A, um texto cifrado C e uma tag de autenticação T como entradas. Como saída, retorna um texto plano P se a tag for válida.

$$\text{Ascon-AEAD128.dec}(K, N, A, C, T) = \begin{cases} P, & \text{se a tag } T \text{ for válida,} \\ \perp, & \text{caso contrário.} \end{cases} \tag{17}$$

No `Ascon-AEAD128`, a chave K é uma chave simétrica estritamente definida e possui um papel ativo e multifásico na construção da segurança do sistema. Ou seja, ela não é utilizada apenas para criptografar o bloco, mas também para construir o próprio estado interno.

Em dispositivos IoT, a proteção da chave depende de mecanismos de segurança que variam conforme a arquitetura do firmware. As normas do NIST exigem que a chave K seja mantida em segredo desde o momento de sua geração até sua destruição definitiva e que não seja utilizada indefinidamente, limitando o tempo de vida útil da credencial.

5.1.1 Criptografia e Descriptografia

Tanto a criptografia quanto a descriptografia no Ascon-AEAD128 operam processando um estado interno de 320 bits, dividido em 128 bits de taxa e 192 bits de capacidade, e são divididas em quatro fases principais.

Inicialização do Estado: Dada uma chave K de 128 bits e um nonce N de 128 bits, o estado interno de 320 bits S é inicializado por meio da concatenação de IV, K e N, em que o valor de inicialização IV é `0x00001000808c0001`.

$$S \leftarrow IV \parallel K \parallel N \tag{18}$$

Em seguida, S é atualizado com a Ascon-p[12]:

$$S \leftarrow \text{Ascon-}p[12](S) \tag{19}$$

Por fim, é realizada a operação XOR entre a chave secreta K e os 128 bits finais do estado interno:

$$S \leftarrow S \oplus (0^{192} \parallel K) \tag{20}$$

É relevante ressaltar que criptanalistas conseguiram atacar somente sete rodadas de permutação, o que deixa uma margem de segurança de cinco rodadas na fase de inicialização. Isso representa um diferencial significativo, tendo em vista que a complexidade do ataque cresce exponencialmente com o aumento do número de rodadas.

Além disso, os ataques existentes contra sete rodadas exigem uma quantidade massiva de dados e poder computacional, frequentemente próxima do limite de 2^{128} operações, tornando-os inviáveis na prática.[1] [3]

[1] Fonte sobre a margem de segurança: https://datatracker.ietf.org/doc/html/draft-ochkas-cose-ascon-01

Processamento de Dados Associados: Essa etapa possui duas partes: a absorção dos dados associados, quando estes não estão vazios, e a aplicação de um bit de separação de domínio ao estado interno. O processo divide-se em duas situações.

- **Se existirem dados associados** (isto é, $|A| > 0$): os dados associados A são divididos em blocos:

$$A_0, A_1, \ldots, A_{m-1}, \widetilde{A_m} \leftarrow \text{parse}(A, 128), \tag{21}$$

em que $m = \lfloor |A|/128 \rfloor$, $|A_i| = 128$ para $0 \leq i \leq m-1$ e $0 \leq |\widetilde{A_m}| < 128$.

Cada bloco de dados associados A_i, para $0 \leq i \leq m$, é absorvido nos primeiros 128 bits do estado:

$$S_{[0:127]} \leftarrow S_{[0:127]} \oplus A_i. \tag{22}$$

Em seguida, a permutação Ascon-p[8] é aplicada ao estado:

$$S \leftarrow \text{Ascon-}p[8](S). \tag{23}$$

O passo final do processamento de dados associados é atualizar o estado com uma constante:

$$S \leftarrow S \oplus (0^{319} \parallel 1) \tag{24}$$

Essa operação provê a separação de domínio.

- **Se não existirem dados associados** (isto é, $|A| = 0$): apenas o último passo descrito em (24) é aplicado.

Processamento do Texto: O texto plano P, incluindo o caso de texto plano vazio, é dividido em blocos:

$$P_0, P_1, \ldots, P_{n-1}, \widetilde{P_n} \leftarrow \text{parse}(P, 128). \tag{25}$$

em que $n = \lfloor |P|/128 \rfloor$, $|P_i| = 128$ para $0 \leq i \leq n-1$ e $|\widetilde{P_n}| = l$, com $0 \leq l \leq 128$.
Se $|P| \bmod 128 = 0$, o último bloco $\widetilde{P_n}$ é vazio.
Para cada P_i, com $0 \leq i \leq n-1$, o estado S é atualizado:

$$S_{[0:127]} \leftarrow S_{[0:127]} \oplus P_i. \tag{26}$$

Em seguida, o bloco de texto cifrado C_i correspondente é gerado:

$$C_i \leftarrow S_{[0:127]}. \tag{27}$$

A permutação Ascon-p[8] é então aplicada ao estado:

$$S \leftarrow \text{Ascon-}p[8](S). \tag{28}$$

Para o último bloco $\widetilde{P_n}$, o estado é atualizado como:

$$S_{[0:127]} \leftarrow S_{[0:127]} \oplus \text{pad}(\widetilde{P_n}, 128). \tag{29}$$

O último bloco de texto cifrado $\widetilde{C_n}$ é obtido como:

$$\widetilde{C_n} \leftarrow S_{[0:l-1]}. \tag{30}$$

O texto cifrado C é construído concatenando cada bloco:

$$C \leftarrow C_0 \parallel \cdots \parallel C_{n-1} \parallel \widetilde{C_n}. \tag{31}$$

Finalização e Geração da Tag: Durante a finalização, primeiramente a chave K é incorporada ao estado S por meio da operação:

$$S \leftarrow S \oplus (0^{128} \parallel K \parallel 0^{64}). \tag{32}$$

Em seguida, o estado S é atualizado utilizando a permutação Ascon-p[12]:

$$S \leftarrow \text{Ascon-}p[12](S). \tag{33}$$

Finalmente, a tag T é gerada por meio da operação XOR entre a chave K e os bits menos relevantes do estado:

$$T \leftarrow S_{[192:319]} \oplus K. \tag{34}$$

O algoritmo retorna o texto cifrado C e a tag T.

5.1.2 Requisitos do AEAD128

Para garantir que o esquema funcione de forma segura, o NIST impõe regras estritas, sendo algumas delas:

1. **Geração de Chaves:** A chave secreta K deve ser gerada de acordo com as recomendações da publicação SP 800-133.

2. **Sigilo da Chave:** A chave secreta K deve ser mantida em sigilo.

3. **Uso de nonce único:** Os nonces devem ser distintos para cada algoritmo de encriptação com uma determinada chav.

4. **Limite máximo de falhas de desencriptação:** Para tags com comprimento λ onde $64 \leq \lambda \leq 128$, o número máximo de falhas de desencriptação toleradas não deve exceder $2^{\lambda-32}$. Para tags mais curtas ($32 \leq \lambda < 64$), o limite de falhas deve ser restrito a 1, embora possa ser flexibilizado até $2^{\lambda-32}$ dependendo de uma análise de risco.

5. **Atualização da chave:** A chave deve obrigatoriamente ser atualizada quando a quantidade de dados atingir o limite de 2^{54} bytes, e é recomendado que seja substituída quando o número de falhas de desencriptação atingir o limite estipulado.

5.2 Ascon-Hash256

O Ascon-Hash256 é um função de hash que utiliza uma mensagem M de tamanho variável como entrada e retorna um digestor de 256 bits. Seu processamento se divide em 3 fases:

1. **Inicialização:** O estado interno de 320 bits do Ascon-Hash256 é inicializado com a concatenação da constante de 64 bits $IV = 0x0000080100cc0002$ e 256 zeros, seguido da permutação Ascon-p[12] de modo que

$$S \leftarrow \text{Ascon-p[12]}(IV \parallel 0^{256}). \tag{35}$$

2. **Absorção da Mensagem:** A fase de absorção é similar a fase processamento de dados associados do Ascon-AEAD128. A mensagem é dividinha em blocos, de modo que

$$M_0, ..., M_{n-1}, \widetilde{M_n} \leftarrow parse(M, 64), \tag{36}$$

onde $|M_i| = 64$ bits para $0 \leq i \leq n-1$ e $0 \leq |\widetilde{M_n}| \leq 63$. O último bloco $\widetilde{M_n}$ pode ser vázio. DEpois, $\widetilde{M_n}$ é preenchido lara criar um bloco inteiro M_n:

$$M_n \leftarrow pad(\widetilde{M_n}, 64). \tag{37}$$

É feito a operação XOR de cada bloco de mensagem M_I com a palavra S, de modo que

$$S_{[0:63]} \leftarrow S_{[0:63]} \oplus M_i, \tag{38}$$

para todos os blocos de mensagem exceto o bloco final M_n, a operação XOR é imediatamente seguida pela aplicação do Asco-p[12] na palavra:

$$S \leftarrow \text{Acon-p[12]}(S). \tag{39}$$

3. **Extração da Hash:** A fase de extração começa após M_n ser absorvido com a aplicação do Ascon-p[12] a palavra:

$$S \leftarrow \text{Acon-p[12]}(S). \tag{40}$$

Depois o valor $S_{[0:63]}$ é pego como um bloco hash de 64 bits H_i, e a palavra é novamente atualizada com Ascon-p[12]:

$$H_i \leftarrow S_{[0:63]} \tag{41}$$

$$S \leftarrow \text{Ascon-p[12]}(S). \tag{42}$$

Os passos (41) e (42) são repetidos alternadamente até os blocos H_0, H_1 e H_2 serem extraido. O bloco hash final é extraido, mas não é aplicado a permutação Ascon-p[12]:

$$H_3 \leftarrow S_{[0:63]}. \tag{43}$$

O digest de 256 bits resultante é a concatenação dos blocos hase de modo que

$$H \leftarrow H_0 \parallel H_1 \parallel H_2 \parallel H_3. \tag{44}$$

Referências

[1] EITC. O que afirma o princípio de kerckhoff? European Information Technology Certification (EITC), 2023. Acessado em: 4 out. 2026.

[2] NIST. Cryptographic key. Computer Security Resource Center (CSRC) Glossary, 2026.

[3] Dmytro Ochkas, Hélène Le Bouder, and Alexander Pelov. Ascon-aead128 for jose and cose. Internet-Draft draft-ochkas-cose-ascon-01, Internet Engineering Task Force, 2026.

[4] Bruce Schneier. *Applied Cryptography: Protocols, Algorithms, and Source Code in C*. John Wiley & Sons, New York, 2nd edition, 1996.

[5] Meltem Sönmez Turan, Kerry McKay, Jinkeon Kang, John Kelsey, and Donghoon Chang. Ascon-based lightweight cryptography standards for constrained devices: Authenticated encryption, hash, and extendable output functions. NIST Special Publication (SP) 800-232, National Institute of Standards and Technology, Gaithersburg, MD, USA, August 2025.